



Caruca v2

LLM-based specification mining for opaque shell commands

caruca_v2 Roadmap

Six phases, what each unlocks, and what gets recorded for the paper

SOURCE `ai_docs/prep/roadmap.md`

GENERATED August 29, 2026 at 7:07 PM

PAGES 11

CARUCA V2 — RESEARCH PROJECT

Table of contents

caruca_v2 Roadmap

Baseline facts this roadmap is built on

The LOC figure that dimension 6 must not be measured against

Phase 1: Baseline Instrumentation

Phase 2: Naive-LLM Baseline

Phase 3: Evaluation Harness

Phase 4: Secure Sandbox

Phase 5: Tool-Augmentation Layer

Phase 6: Web GUI (optional, deferred)

Evaluation-Dimension Coverage Audit

Self-Critique

caruca_v2 Roadmap

Produced via the `10_generate_build_order_worker` planning pass. Reads on from `master_idea.md`, `component_functionality.md`, `data_telemetry_schema.md`, and `system_architecture.md`.

Phases are MVP components built end to end (implementation, telemetry, validation, documentation), not technical layers. Phases 1-3 are Tier 0 and Phases 4-6 are Tier 1, per `memory/caruca_v2_build_tiering.md`. The phase order was settled by that tiering decision; this document adds what the tiering did not specify: per-phase exit criteria, an evaluation-dimension coverage audit, and per-phase recording obligations for the paper resubmission.

Baseline facts this roadmap is built on

Measured against v1 at `~/stevens/caruca/`, not assumed:

- **Assets available for reuse:** 121 curated ground-truth syntax specs, 54 committed reference PaSh annotations, 13 benchmark ground-truth annotations, plus `eval/cmp_specs.py` and the real-world invocation corpus.
- **generate, annotate, and oracle work today;** `syntax-spec` does not (DSPy 3.3.1 import failure).
- **No cost or wall-clock telemetry exists anywhere.** `eval/performance/` holds invocation counting, not timing. Dimension 2 starts from zero, which is what makes Phase 1 the first phase.
- **A second, independent correctness methodology exists** beyond `cmp_specs.py`: `eval/syntax-spec-correctness.sh` diffs ASP atoms via `clingo` against `benchmarks/gnu-coreutils/asp/*.pl` for 5 commands (`rm`, `mv`, `cp`, `cat`, `mkdir`). It needs `clingo` and `jq`, and is also blocked on the broken `syntax-spec`. Useful as a cross-check on Phase 3's numbers for those 5 commands.
- **v1 never explores model or decoding-parameter choice for its one LLM step.** `llm.py` hardcodes `gpt-4o`, sets `seed=42` but no explicit temperature, and extracts code via a fragile regex on fenced code blocks rather than a structured-output mode. No alternative model, temperature, or output-format constraint was ever tried, for v1 or for anything else.

The LOC figure that dimension 6 must not be measured against

The paper reports 6,520 LOC of Python. Measured by module today, that total is approximately:

- Hand-written pipeline logic: **~3,456 LOC** (`ir` 1,249, `tracer` 946, `annotator` 586, `cli` 354, plus package-level modules)
- Generated syntax-spec data: **~3,130 LOC** across `syntax_specs/`

The two sum to roughly the published figure. This matters because dimension 6 is “reduction in hand-encoded logic,” and roughly half the headline number is LLM-generated spec data, not hand-encoded logic. Any dimension-6 claim should be stated against the ~3,456 LOC of pipeline logic, with the split disclosed. Measuring against 6,520 would overstate the reduction by about 2x.

Phase 1: Baseline Instrumentation

Goal: v1’s own LLM step becomes measurable for cost, tokens, latency, and model ID for the first time. This is a number the original paper never reported.

Depends on: v1’s existing `syntax-spec` step. Restoring it against DSPy 3.3.1 is in scope for this phase, not a separate phase. Reads `data_telemetry_schema.md` for the telemetry record shape.

Work: Migrate `llm.py` off the removed DSPy 2.x APIs (`dspy.OpenAI` , `dspy.Suggest` , `dspy.predict.Retry` , `assert_transform_module`), pin the dependency, and make the hardcoded model configurable. Wrap the call to capture the telemetry record and write it as a sidecar JSON. Serialize or isolate calls, because `dspy.settings.configure` rebinds the LM process-wide and would otherwise corrupt concurrent runs at different models or seeds.

Exit criteria: `caruca syntax-spec CMD` runs again; a telemetry sidecar is produced per run with every field in the schema populated; regenerating a spec for a command with existing ground truth produces a diffable result.

Validation: Unlocks dimension 2 (cost/performance) for v1.

Paper-readiness note: Record the model ID, the DSPy version before and after, and the exact v1 commit. Because the paper’s numbers are GPT-4o numbers, any later correctness comparison has to separate “better model” from “better method,” and that separation is only possible if this phase records what it ran.

Phase 2: Naive-LLM Baseline

Goal: A second system exists to compare against v1 at all — and its model/configuration choices are selected deliberately, not left as unexamined defaults.

Depends on: Phase 1 (for the telemetry record shape proven in practice, and for a working v1 spec to compare against). Reads `component_functionality.md` for I/O and CLI surface.

Why this includes model/configuration selection, not just a prompt: v1's behavior is fixed by hand-written code, so it has no configuration surface at all outside its one LLM call. The moment v2 replaces hardcoded logic with an NLP-based component, model choice, decoding parameters (e.g. temperature), and output-format constraints (e.g. structured/JSON output versus free text) stop being incidental setup and become part of the method itself — they directly drive accuracy, consistency, and cost, the same way choosing an algorithm did in v1's code. Selecting and documenting them is therefore core scope for this phase, not an optional enhancement layered on top of it.

Work: A single, few-shot-primed prompt taking a command's binary name plus its documentation and emitting a spec in v1's Python-DSL shape. No agentic tool use, no validation or retry loop. Emits the same telemetry record shape as Phase 1. The prompt itself stays deliberately simple, per the scoping guidance in

`memory/caruca_v2_baseline_scope.md` — what's new is that the model, its decoding parameters, and its output-format constraint are chosen through a small, fixed comparison rather than picked arbitrarily.

Before freezing this baseline's exact configuration, compare model choice, decoding parameters, structured output (schema-constrained/JSON output versus free text plus regex extraction — the latter being the same fragile pattern v1's `extract_code()` uses) and 2-3 prompt-phrasing variants aimed at run-to-run consistency, on a held-out command subset (not the full corpus). Freeze the configuration this comparison supports, then run it unchanged for every command in Phase 3's actual comparison. This is a fixed, one-time selection step, not iterative tuning of the baseline's output to win the comparison — that distinction matters and is worth stating explicitly wherever this phase is described, since “no tuning” (the scoping guidance's actual constraint) means “not iteratively refined to produce a better answer,” not “configured without deliberation.”

Exit criteria: Produces a syntactically valid spec in v1's DSL for a target command set; telemetry sidecar per run; the plain-docs condition is labeled distinctly from any future augmented condition; the model/configuration comparison's results (every variant tried, and why the final configuration was chosen) are written up before the frozen baseline's full-corpus run begins.

Validation: Unlocks dimension 2 for v2, and makes dimension 1 possible in Phase 3. Also produces a standalone finding, parallel to the cost and repeatability gaps already identified: neither v1 nor the originally-scoped naive baseline ever varied model choice, decoding parameters, or output-format constraints — because v1’s hardcoded design never needed to, and a naive baseline’s default config is exactly the kind of choice that’s easy to leave unexamined. This is the first time it’s characterized, for either system.

Paper-readiness note: Record the exact prompt text, the few-shot examples used, model ID, temperature, and seed — for the frozen baseline **and** for every configuration compared, not just the winner. The prompt is the method here, so an unrecorded prompt makes the result uncitable, and a resubmission claim about model/parameter choice is only as strong as the record of what else was tried. Framing note for whoever writes this up: state plainly that configuration selection is a first-class part of moving from a hardcoded to an NLP-based approach, not an afterthought bolted onto the baseline.

Phase 3: Evaluation Harness

Goal: The core deliverable. The three-way comparison (v1, naive-LLM, ground truth) and the percent-change roll-up become possible for the first time.

Depends on: Phases 1 and 2. Reads `data_telemetry_schema.md` for the comparison-result schema.

Work: A typed `sweep / compare` function with the CLI as a thin wrapper, so it is callable by an LLM agent today without a bespoke tool integration. Reuses `cmp_specs.py` for the command-level exact-match tally. Fixes the element-versus-field denominator so an argument-level percentage comparable to the paper’s 99.7% figure can be reported, and tags which method produced each number. Adds repeated-sampling variance runs across both systems. Aggregates JSON sidecars into the derived SQLite layer for roll-up queries.

Exit criteria: A single command produces a per-command comparison record and a corpus-level roll-up; variance is reported with a sample count; coverage numbers are disaggregated rather than blended; results are reproducible from the recorded run manifest.

Validation: Unlocks dimension 1 (correctness, Q2-style), dimension 4 (consistency/reproducibility, which the original paper never measured), and dimension 5 (percent-change roll-up).

Paper-readiness note: This phase produces the tables. Record the methodology per number, including which of the three correctness methods produced it, the sample count behind every variance figure, and the strong-normalization-only subtotal alongside any blended coverage percentage. Cross-check the 5 commands covered by `eval/syntax-spec-correctness.sh` against its independent ASP-based method.

Phase 4: Secure Sandbox

Goal: Coverage extends past v1's non-destructive, man-page-documented sweet spot, and the annotation-diff comparison becomes possible.

Depends on: Phase 3, and on resolving the backend decision that `system_architecture.md` leaves open. Requires the provisioned host; this phase cannot run on the current WSL machine.

Work: Spike the isolation backend first (Docker/overlayfs extension, Firecracker, or gVisor), gated on preserving strace/ptrace visibility. Then build the two fixture profiles: `v1-faithful` as the apples-to-apples control, and `v2-extended` adding symlinks, permission states, deeper directory trees, correlated multi-file content, pipe/SIGPIPE cases, empty content, wider integers, and environment variables. Extend `FSInteraction` and the annotator's path classification for the new trace shapes. Results are tagged by profile, never blended.

Exit criteria: Both profiles produce `Traces` JSON that v1's annotator consumes; the `v1-faithful` profile reproduces v1's own numbers closely enough to serve as a control; extended-shape traces either annotate correctly or are recorded as coverage-gap findings.

Validation: Unlocks dimension 3 (coverage) and extends dimension 1 to annotation-level comparison.

Paper-readiness note: Record the backend chosen and why, the hardware, and the `v1-faithful` control numbers next to v1's published ones. Timing comparisons against the paper's cost section are only meaningful on comparable hardware. Restore a minimal annotation test before touching annotator logic; no currently-passing test exercises annotation output.

Phase 5: Tool-Augmentation Layer

Goal: Commands outside the man-page-documented sweet spot become reachable.

Depends on: Phase 2 (the baseline it augments) and Phase 3 (to measure whether augmentation helps).

Work: A starter toolset covering binary and format inspection, static analysis, and help-string extraction, feeding an augmented documentation context into the naive-LLM baseline through an explicit flag. Runs as a separately labeled condition, never merged into the plain-docs baseline numbers.

Exit criteria: At least one command that the plain-docs baseline cannot handle produces a usable spec under augmentation, with both conditions reported side by side.

Validation: Extends dimension 3 (coverage) along the documentation axis rather than the execution axis.

Paper-readiness note: Record per-tool telemetry and which tools fired for each command. The claim here is about reach, so the evidence has to show what was unreachable before.

Phase 6: Web GUI (optional, deferred)

Goal: Interactive side-by-side comparison instead of CLI-only use.

Depends on: Phases 1-5 existing as CLI subcommands. Adds no new data format.

Work: A PTY-backed wrapper that spawns the existing CLI subcommands and streams them to two xterm.js panes, with a metrics panel reading the telemetry sidecars those subcommands already write.

Exit criteria: Both systems run visibly side by side for a single command, with metrics drawn from existing telemetry rather than recomputed.

Validation: Unlocks no new evaluation dimension. This is a presentation and demonstration capability.

Paper-readiness note: Not a source of results. Useful for advisor demonstrations and possibly a figure.

Evaluation-Dimension Coverage Audit

DIMENSION	UNLOCKED BY	STATUS
1. Correctness/fidelity	Phase 3 (Q2-style), Phase 4 (annotation-diff)	Covered
2. Cost/performance	Phase 1 (v1 side), Phase 2 (v2 side + model/parameter sensitivity sweep)	Covered
3. Coverage	Phase 4 (execution axis), Phase 5 (documentation axis)	Covered
4. Consistency/reproducibility	Phase 2 (sweep tests which model/params/prompting reduce variance), Phase 3 (repeated-sampling variance on the frozen config)	Covered
5. Percent-change roll-up	Phase 3	Covered
6. Reduction in hand-encoded logic	No phase	Partially unreachable, see below

Dimension 6 does not have a phase, and mostly cannot have one in Tier 0 or Tier 1. The naive-LLM baseline substitutes only for v1's syntax-spec step. Every downstream stage, including config generation, tracing, and annotation, is v1's code being reused rather than replaced, and Phase 4 and Phase 5 add hand-written logic rather than removing it. The honest scope of dimension 6 within this roadmap is narrow: how much of `llm.py`'s DSPy scaffolding, retry logic, and prompt engineering a single prompt replaces, in LOC, measured against the ~3,456 LOC of pipeline logic rather than the published 6,520.

The full dimension-6 claim requires the deferred agentic rebuild, where downstream stages are actually replaced. Recommendation: record the narrow measurement as part of Phase 2's documentation, and state plainly in any write-up that the broad claim is out of scope until the agentic rebuild exists. Silently reporting a narrow number as if it were the broad one is the failure mode to avoid.

Self-Critique

Strengths

- Every phase is a component built end to end, not a technical layer, and each names a capability that did not exist before it.
- Prerequisites hold in order: nothing depends on something built later, and the two phases requiring the provisioned host (4 and 5) come after everything that runs locally.
- The highest-uncertainty component, the sandbox backend, blocks the least work. Phases 1-3 deliver the core comparison without it.
- Paper-readiness is built into every phase rather than bolted on at the end.
- Phase language frames v2 as extending v1: reusing its annotator, its ground truth, and its methodology.

Issues Found

- 🚨 Dimension 6 has no owning phase, and the coverage audit above shows this is not a sequencing oversight but a scope reality. Surfaced explicitly rather than papered over, with a narrow measurement assigned to Phase 2 and the broad claim deferred.
- ⚠️ Phase 4 is the largest phase by a wide margin: a backend spike, two fixture profiles, and annotator extension work. It is a candidate to split into 4a (backend spike plus `v1-faithful` control) and 4b (`v2-extended` profile plus annotator extension) once the spike's result is known. Left unsplit here because the right boundary depends on what the spike finds.
- ⚠️ Phase 1 bundles a dependency migration with a new capability. This is deliberate, since telemetry cannot be captured from a step that does not run, but it means Phase 1 carries more risk than its size suggests.
- ⚠️ The target command set for Phase 2 is unspecified. The 121 available ground-truth specs make the full corpus possible, but a smaller starting set would validate the pipeline faster. Left open as an implementation decision.
- ⚠️ Model/configuration selection in Phase 2 is in scope, not scope creep — it's an inherent part of moving from v1's hardcoded pipeline to an NLP-based one. Keep it bounded anyway (a small held-out subset, a short fixed list of variants decided in advance, not an open-ended search), so the distinction from iterative tuning of the baseline's output stays clean and defensible.

Revisions Made

- Added the LOC-split finding, which changes what dimension 6 can honestly be measured against.
- Added `eval/syntax-spec-correctness.sh` as an independent cross-check on Phase 3, having found it during codebase analysis.

- Added explicit exit criteria to every phase, since the tiering decision defined scope but not completion.
- Moved the annotation-test prerequisite into Phase 4's paper-readiness note, since no passing test currently exercises annotation output.
- Added model/decoding-parameter/output-format selection to Phase 2 as core scope, not an add-on: moving from v1's hardcoded pipeline to an NLP-based component makes these choices a first-class part of the method, not incidental setup. Extends dimension 2 and dimension 4 coverage and produces its own citable finding, while keeping the distinction from iterative output-tuning explicit.